

DM I: Block 'Model Evaluation and Model Comparisons'

Unit 1: Basics on classifier evaluation

Myra Spiliopoulou



FACULTY OF
COMPUTER SCIENCE



- 1 Evaluation workflow
- 2 Confusion matrix
- 3 and Quality Functions
- 4 Workflow+ : Sampling for testing
- 5 Workflow++ : Combining classifiers
- 6 Imbalanced classes
- 7 Closing

RECALL from the 'Classification' Unit 'Underpinnings'

Classification – Learning Phase

A classification algorithm takes as input a set of mutually exclusive classes

$$\mathcal{C} = \{C_1, \dots, C_l\}$$

and, for each $C_i \in \mathcal{C}$, $i = 1 \dots l$, a set of instances D_i belonging to C_i .^a

The set $D = \cup_{i=1}^l D_i$ is used for *learning*, i.e. for training.

D must be representative of the population $\mathcal{D}$ under study.

RECALL from the 'Classification' Unit 'Underpinnings'

Classification – Learning Phase

A classification algorithm takes as input a set of mutually exclusive classes

$$\mathcal{C} = \{C_1, \dots, C_l\}$$

and, for each $C_i \in \mathcal{C}, i = 1 \dots l$, a set of instances D_i belonging to C_i .^a

The set $D = \cup_{i=1}^l D_i$ is used for *learning*, i.e. for training.

D must be representative of the population $\mathcal{D}$ under study.

We split D into a *training set* D_{train} to be used for model induction and a *test set* D_{test} to be used for model deduction (includes: evaluation).

RECALL from the 'Classification' Unit 'Underpinnings'

Classification – Learning Phase

A classification algorithm takes as input a set of mutually exclusive classes

$$\mathcal{C} = \{C_1, \dots, C_l\}$$

and, for each $C_i \in \mathcal{C}$, $i = 1 \dots l$, a set of instances D_i belonging to C_i .^a

The set $D = \cup_{i=1}^l D_i$ is used for *learning*, i.e. for training.

D must be representative of the population $\mathcal{D}$ under study.

We split D into a *training set* D_{train} to be used for model induction and a *test set* D_{test} to be used for model deduction (includes: evaluation).

The classification algorithm builds a *classifier* ξ ,
i.e. a model that reflects what makes the instances in each class different from the instances in the other classes.

^a**NOTE:** We now denote the number of classes as l . Earlier, we used k . We see later why another symbol is used.

EXAMPLE

On the dataset:

Id	I2	I15	I30	I22	I31	I9	I26	Response
#1	f	VH	yes	better	no	r	no	yes
#2	m	M	no	better	no	b	yes	no
#3	m	M	no	worse	no	b	no	no
#4	f	VH	yes	worse	no	b	no	yes
#5	m	L	no	no effect	no	l	no	no
#6	m	M	no	better	no	l	no	no
#7	f	VH	yes	better	yes	l	yes	yes
#8	f	H	no	better	yes	r	no	no
#9	f	H	yes	better	no	l	no	yes
#10	m	M	yes	worse	no	b	no	no
#11	m	M	no	no effect	no	l	no	no
#12	f	H	no	no effect	no	r	no	no
#13	f	L	yes	better	no	l	yes	yes
#14	m	M	no	worse	no	b	no	no
#15	m	L	no	no effect	no	l	yes	no

we have built following classifier M :

```
IF I2=m THEN Response=no
ELSEIF I26=no THEN Response=yes
ELSE Response=no
```

How to compute the quality of this classifier?

EXAMPLE continued

To evaluate the classifier, we have this dataset:

Id	I2	I15	I30	I22	I31	I9	I26	Response
#91	f	VH	yes	better	no	r	no	yes
#92	m	M	no	better	no	b	yes	yes
#93	m	M	no	worse	no	b	yes	no
#94	f	VH	yes	worse	no	b	yes	yes
#95	m	L	no	no effect	no	l	no	no
#96	m	M	no	better	no	l	no	no
#97	f	VH	yes	better	yes	l	no	yes
#98	f	H	no	better	yes	r	yes	no
#99	f	H	yes	better	no	l	no	yes

- 1 Evaluation workflow
- 2 Confusion matrix**
- 3 and Quality Functions
- 4 Workflow+ : Sampling for testing
- 5 Workflow++ : Combining classifiers
- 6 Imbalanced classes
- 7 Closing

Classifier Evaluation

Confusion matrix for a binary classification problem:

		Classifier assignment	
		<i>Class POS</i>	<i>Class NEG</i>
True membership of instances to classes	<i>Class POS</i>	true positives	false negatives
	<i>Class NEG</i>	false positives	true negatives

Classifier Evaluation

Confusion matrix for a binary classification problem:

		Classifier assignment	
		<i>Class 1</i>	<i>Class 0</i>
True membership of instances to classes	<i>Class 1</i>	f_{11}	f_{10}
	<i>Class 0</i>	f_{01}	f_{00}

- 1 Evaluation workflow
- 2 Confusion matrix
- 3 and Quality Functions**
- 4 Workflow+ : Sampling for testing
- 5 Workflow++ : Combining classifiers
- 6 Imbalanced classes
- 7 Closing

Classifier Evaluation

Quality measurement functions on the basis of the confusion matrix:

Accuracy

- *Accuracy* for the binary classification problem: $\frac{f_{11}+f_{00}}{f_{11}+f_{10}+f_{01}+f_{00}}$
- *Accuracy* for an arbitrary classification problem: $\frac{\#hits}{\#testInstances}$

Classifier Evaluation

Quality measurement functions on the basis of the confusion matrix:

Accuracy

- ▶ *Accuracy* for the binary classification problem: $\frac{f_{11}+f_{00}}{f_{11}+f_{10}+f_{01}+f_{00}}$
- ▶ *Accuracy* for an arbitrary classification problem: $\frac{\#hits}{\#testInstances}$

Quality functions for the class POS (class 1)

- ▶ $precision = \frac{f_{11}}{f_{11}+f_{01}}, recall = \frac{f_{11}}{f_{11}+f_{10}}$
- ▶ F-measure $F = \frac{2 \cdot precision \cdot recall}{precision + recall}$
and generalization $F_{\beta} = \frac{(1+\beta^2)f_{11}}{(1+\beta^2)f_{11}+\beta^2f_{10}+f_{01}}$

Sensitivity (recall wrt Class 1) and Specificity (recall wrt Class 0)

$$sensitivity = \frac{f_{11}}{f_{11}+f_{10}}, specificity = \frac{f_{00}}{f_{00}+f_{01}}$$

Scaling up the Confusion Matrix

RECALL: A classification algorithm takes as input a set of mutually exclusive classes $\mathcal{C} = \{C_1, \dots, C_l\}$

and, for each $C_i \in \mathcal{C}, i = 1 \dots l$, a set of instances D_i belonging to C_i .

1↓

Confusion matrix for a l -nary classification problem:

		Classifier assignment				
		Class 1	Class 2	Class 3	...	Class l
True membership of instances to classes	Class 1	f_{11}	f_{12}	f_{13}	...	f_{1l}
	Class 2	f_{21}	f_{22}	f_{23}	...	f_{2l}
	Class 3	f_{31}	f_{32}	f_{33}	...	f_{3l}
	...	...	...	...	...	...
	Class l	f_{l1}	f_{l2}	f_{l3}	...	f_{ll}

What if the target variable is ordinal?

Example: Response $\in \{\text{better, no effect, worse}\}$

What if the target variable is ordinal?

Example: Response $\in \{\text{better, no effect, worse}\}$

Map labels to ordinal numbers $\rightarrow$ better:2, no effect:1, worse:0

What if the target variable is ordinal?

Example: Response $\in \{\text{better, no effect, worse}\}$

Map labels to ordinal numbers $\rightarrow$ better:2, no effect:1, worse:0

Approach

Let $OL = \{1, 2, \dots, l\}$ be the set of values of the target variable (without loss of generality), and let D_{test} be the testset. For $x \in D_{test}$, let y_x be the true label and let $\hat{y}_x$ be the label predicted by the model.

What if the target variable is ordinal?

Example: Response $\in \{\text{better, no effect, worse}\}$

Map labels to ordinal numbers $\rightarrow$ better:2, no effect:1, worse:0

Approach

Let $OL = \{1, 2, \dots, l\}$ be the set of values of the target variable (without loss of generality), and let D_{test} be the testset. For $x \in D_{test}$, let y_x be the true label and let $\hat{y}_x$ be the label predicted by the model.

- ▶ Root Mean Square Error (RMSE): $\sqrt{\frac{1}{|D_{test}|} \sum_{x \in D_{test}} (y_x - \hat{y}_x)^2}$
- ▶ Mean Absolute Error (MAE): $\frac{1}{|D_{test}|} \sum_{x \in D_{test}} |y_x - \hat{y}_x|$

What if the target variable is ordinal?

Example: Response $\in \{\text{better, no effect, worse}\}$

Map labels to ordinal numbers $\rightarrow$ better:2, no effect:1, worse:0

Approach

Let $OL = \{1, 2, \dots, l\}$ be the set of values of the target variable (without loss of generality), and let D_{test} be the testset. For $x \in D_{test}$, let y_x be the true label and let $\hat{y}_x$ be the label predicted by the model.

- ▶ Root Mean Square Error (RMSE): $\sqrt{\frac{1}{|D_{test}|} \sum_{x \in D_{test}} (y_x - \hat{y}_x)^2}$
- ▶ Mean Absolute Error (MAE): $\frac{1}{|D_{test}|} \sum_{x \in D_{test}} |y_x - \hat{y}_x|$

RMSE and Accuracy

Sometimes, RMSE is referred to as 'accuracy' when the context (i.e. ordinal target variable) is clear. What is the relationship between accuracy and RMSE?

- 1 Evaluation workflow
- 2 Confusion matrix
- 3 and Quality Functions
- 4 Workflow+ : Sampling for testing**
- 5 Workflow++ : Combining classifiers
- 6 Imbalanced classes
- 7 Closing

RECALL: trainingset and testset on our running example

D_{train} :

Id	I2	I15	I30	I22	I31	I9	I26	Response
#1	f	VH	yes	better	no	r	no	yes
#2	m	M	no	better	no	b	yes	no
#3	m	M	no	worse	no	b	no	no
#4	f	VH	yes	worse	no	b	no	yes
#5	m	L	no	no effect	no	l	no	no
#6	m	M	no	better	no	l	no	no
#7	f	VH	yes	better	yes	l	yes	yes
#8	f	H	no	better	yes	r	no	no
#9	f	H	yes	better	no	l	no	yes
#10	m	M	yes	worse	no	b	no	no
#11	m	M	no	no effect	no	l	no	no
#12	f	H	no	no effect	no	r	no	no
#13	f	L	yes	better	no	l	yes	yes
#14	m	M	no	worse	no	b	no	no
#15	m	L	no	no effect	no	l	yes	no

 D_{test} :

Id	I2	I15	I30	I22	I31	I9	I26	Response
#91	f	VH	yes	better	no	r	no	yes
#92	m	M	no	better	no	b	yes	yes
#93	m	M	no	worse	no	b	yes	no
#94	f	VH	yes	worse	no	b	yes	yes
#95	m	L	no	no effect	no	l	no	no
#96	m	M	no	better	no	l	no	no
#97	f	VH	yes	better	yes	l	no	yes
#98	f	H	no	better	yes	r	yes	no
#99	f	H	yes	better	no	l	no	yes

There are many ways to split D into D_{train} and D_{test}

Sampling the test data

[Ch.3, S. 3.6.1, p. 185]

Holdout method

Split the learning sample on a training dataset and a test dataset; learn a model on the training data; test the model on the test data.

Limitations:

- ▶ *Too few data for learning:* The more data are used for testing, the more reliable is the performance estimation, but the more data are missing for learning.
- ▶ *Interdependence of training & test dataset:* If a class is underrepresented in the training dataset, it will be overrepresented in the test dataset (and vice versa).

Sampling the test data

[Ch.3, S. 3.6.1, p. 185]

Holdout method

Split the learning sample on a training dataset and a test dataset; learn a model on the training data; test the model on the test data.

Limitations:

- ▶ *Too few data for learning:* The more data are used for testing, the more reliable is the performance estimation, but the more data are missing for learning.
- ▶ *Interdependence of training & test dataset:* If a class is underrepresented in the training dataset, it will be overrepresented in the test dataset (and vice versa).

Random subsampling

Invoke the holdout method k times, compute the model performance f_i at each invocation, and then derive the overall performance as $\frac{1}{k} \sum_{i=1}^k f_i$.

Sampling over *all* data

[Ch. 3, S. 3.6.2, p. 185]

Cross validation

Core idea:

- ▶ use each record $k - 1$ times for training and once for testing
- ▶ aggregate the error values (resp. performance values) over all k tests

Sampling over *all* data

[Ch. 3, S. 3.6.2, p. 185]

Cross validation

Core idea:

- ▶ use each record $k - 1$ times for training and once for testing
- ▶ aggregate the error values (resp. performance values) over all k tests

k -fold cross validation:

- ▶ split the learning dataset into k equi-sized *folds* $D_1, \dots, D_k$ ^a
- ▶ for $i = 1, \dots, k$: use the $k-1$ folds $\cup_{j \neq i} D_j$ for training and the fold D_i for testing
- ▶ aggregate the error values (resp. performance values) over all k tests

Sampling over *all* data

[Ch. 3, S. 3.6.2, p. 185]

Cross validation

Core idea:

- ▶ use each record $k - 1$ times for training and once for testing
- ▶ aggregate the error values (resp. performance values) over all k tests

k -fold cross validation:

- ▶ split the learning dataset into k equi-sized *folds* $D_1, \dots, D_k$ ^a
- ▶ for $i = 1, \dots, k$: use the $k-1$ folds $\cup_{j \neq i} D_j$ for training and the fold D_i for testing
- ▶ aggregate the error values (resp. performance values) over all k tests

Leave-one-out: k -fold cross validation over learning set D , with $k = |D|$

^aSplit D into $D_1, \dots, D_k$ so that $D = \cup_{i=1}^k D_i$ and $\forall i \neq j : D_i \cap D_j = \emptyset$

Sampling over *all* data

Bootstrap

mentioned in [Ch. 3, S. 3.10, p. 200]

Core idea:

- ▶ for $i = 1, \dots, k$: sample $|D|$ instances to build the training set, use the remaining instances for testing
- ▶ aggregate the error values (resp. performance values) over all k tests

- 1 Evaluation workflow
- 2 Confusion matrix
- 3 and Quality Functions
- 4 Workflow+ : Sampling for testing
- 5 Workflow++ : Combining classifiers**
- 6 Imbalanced classes
- 7 Closing

Combining classifiers to improve classification quality

Combining classifiers into an *ensemble*

Each of the m ensemble members may be built by

- ▶ a different classification algorithm (e.g. tree induction algorithm, NB)
- ▶ the same classification algorithm, different parameter settings
- ▶ the same classification algorithm and setting, different preparation steps
- ▶ exactly the same classification algorithm, parameter settings and inputs:
 - ▶ *Random forest*: ensemble composed of m DTs
 - ▶ *Boosting*: each record has an initial weight; the weights of the records misclassified by classifier ζ_i are increased to train the classifier ζ_{i+1} .

Combining classifiers to improve classification quality

Combining classifiers into an *ensemble*

Each of the m ensemble members may be built by

- ▶ a different classification algorithm (e.g. tree induction algorithm, NB)
- ▶ the same classification algorithm, different parameter settings
- ▶ the same classification algorithm and setting, different preparation steps
- ▶ exactly the same classification algorithm, parameter settings and inputs:
 - ▶ *Random forest*: ensemble composed of m DTs
 - ▶ *Boosting*: each record has an initial weight; the weights of the records misclassified by classifier ζ_i are increased to train the classifier ζ_{i+1} .

Corporate decision in an ensemble

The label of each record in the test set is decided by:

- simple majority voting
- majority voting – each vote weighted by the ensemble member's confidence

and the quality of the ensemble is computed on the basis of the *corporate decisions* for the records of the test set.

- 1 Evaluation workflow
- 2 Confusion matrix
- 3 and Quality Functions
- 4 Workflow+ : Sampling for testing
- 5 Workflow++ : Combining classifiers
- 6 Imbalanced classes**
- 7 Closing

Class imbalance

[see also Ch. 6, S. 6.11]

One class is much more rare than the other(s), e.g.

- ▶ Intrusion detection
- ▶ Fraud detection
- ▶ Distinguishing between people suffering from a disease and those that do not
- ▶ Identifying the sentiment in each of a set of opinionated documents, when some types of sentiment occur rarely

The cost of a wrong class attribution varies substantially among the classes, e.g.

- ▶ Misclassifying a healthy person as ill vs misclassifying an ill person as healthy

In many cases, both occur together for some class.

1. How to **evaluate** when the classes are imbalanced

In some cases, the F-measure can alleviate the pitfall of accuracy.

Slideset of Chapter 4, subchapter “Imbalanced classes”

SLIDES 8 (F-measure), 9-12 (F-measure vs accuracy for different class distributions)

Alternative Measures

	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	a	b
	Class=No	c	d

$$\text{Precision (p)} = \frac{a}{a + c}$$

$$\text{Recall (r)} = \frac{a}{a + b}$$

$$\text{F - measure (F)} = \frac{2rp}{r + p} = \frac{2a}{2a + b + c}$$

Alternative Measures

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
	Class=Yes	Class=No
	10	0
	10	980

$$\text{Precision (p)} = \frac{10}{10+10} = 0.5$$

$$\text{Recall (r)} = \frac{10}{10+0} = 1$$

$$\text{F-measure (F)} = \frac{2*1*0.5}{1+0.5} = 0.62$$

$$\text{Accuracy} = \frac{990}{1000} = 0.99$$

Alternative Measures

	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	10	0
	Class=No	10	980

$$\text{Precision (p)} = \frac{10}{10+10} = 0.5$$

$$\text{Recall (r)} = \frac{10}{10+0} = 1$$

$$\text{F-measure (F)} = \frac{2*1*0.5}{1+0.5} = 0.62$$

$$\text{Accuracy} = \frac{990}{1000} = 0.99$$

	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	1	9
	Class=No	0	990

$$\text{Precision (p)} = \frac{1}{1+0} = 1$$

$$\text{Recall (r)} = \frac{1}{1+9} = 0.1$$

$$\text{F-measure (F)} = \frac{2*0.1*1}{1+0.1} = 0.18$$

$$\text{Accuracy} = \frac{991}{1000} = 0.991$$

Alternative Measures

	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	40	10
	Class=No	10	40

Precision (p) = 0.8

Recall (r) = 0.8

F - measure (F) = 0.8

Accuracy = 0.8

Alternative Measures

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
	40	10
ACTUAL CLASS	Class=Yes	Class=No
	10	40

Precision (p) = 0.8

Recall (r) = 0.8

F - measure (F) = 0.8

Accuracy = 0.8

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
	40	10
ACTUAL CLASS	Class=Yes	Class=No
	1000	4000

Precision (p) $\approx$ 0.04

Recall (r) = 0.8

F - measure (F) $\approx$ 0.08

Accuracy $\approx$ 0.8

2. How to **learn** when the classes are imbalanced?

- ▶ Induce classification rules and prioritize the rules of the minority class
 RIPPER: Repeated Incremental Pruning to Produce Error Reduction ¹
- ▶ Sample from the training data so that the classes become balanced
 - ▶ Oversample from the minority class SMOTE ² and followups
 - ▶ Undersample from the majority class
 - ▶ Combination of the oversampling and undersampling
 “local over-sampling of difficult examples from the minority class with removing or relabeling noisy examples from the majority classes.” ³

¹Cohen, W. W. (1995). Fast effective rule induction. In Machine Learning Proc. 1995 (pp. 115-123). Morgan Kaufmann.

²Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. J. of Artificial Intelligence Research, 16, 321-357.

³Stefanowski, J., & Wilk, S. (2008). Selective pre-processing of imbalanced data for improving classification performance. In Int. Conf. on Data Warehousing and Knowledge Discovery (pp. 283-292). Springer, Berlin, Heidelberg.

Cost of misclassification and over/under-sampling

Figure 6 from SMOTE⁴ on experiment design, including (a) comparison to RIPPER with undersampling and (b) coupling with RIPPER

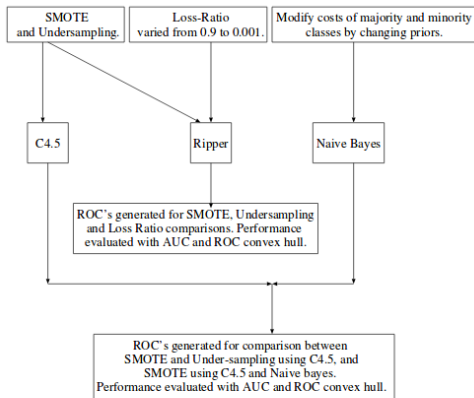


Figure 6: Experiments Overview

⁴Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. J. of Artificial Intelligence Research, 16, 321-357.

- 1 Evaluation workflow
- 2 Confusion matrix
- 3 and Quality Functions
- 4 Workflow+ : Sampling for testing
- 5 Workflow++ : Combining classifiers
- 6 Imbalanced classes
- 7 Closing**

Progress and outlook

We have seen:

- ✓ different ways of modeling classifier quality
- ✓ methods for building training & test samples by sampling (a) without replacement and (b) with replacement
- ✓ ways of combining classifiers to improve classification quality
- ✓ ways of dealing with class imbalance

We have seen many ways of building a single/ensemble classifier:

- ▶ How to figure out which classifier is the best one?

Thank you very much!

Questions?